

λέξις

a Quarto theme

John Paul Helveston

2026-07-08

Text styling

Header level 1

Header level 2

Header level 3

Header level 4

Header level 5

Header level 6

Regular

Italics

Bold

Bold italics

~~Strikethrough~~

Fancy text

[external link](#)

`Inline code`

Inverse text styling

Header level 1

Header level 2

Header level 3

Header level 4

Header level 5

Header level 6

Regular

Italics

Bold

Bold italics

~~Strikethrough~~

Fancy text

[external link](#)

`Inline code`

Colors!

Use this...

- `[text]{.red}`
- `[text]{.orange}`
- `[text]{.yellow}`
- `[text]{.green}`
- `[text]{.darkgreen}`
- `[text]{.blue}`
- `[text]{.darkblue}`
- `[text]{.purple}`
- `[text]{.black}`

...to get this

- **text**
- **text**
- **text**
- **text**
- **text**
- **text**
- **text**
- **text**
- **text**

Tables

manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
audi	a4	1.8	1999	4	auto(l5)	f	18	29	p	compac
audi	a4	1.8	1999	4	manual(m5)	f	21	29	p	compac
audi	a4	2.0	2008	4	manual(m6)	f	20	31	p	compac
audi	a4	2.0	2008	4	auto(av)	f	21	30	p	compac
audi	a4	2.8	1999	6	auto(l5)	f	16	26	p	compac
audi	a4	2.8	1999	6	manual(m5)	f	18	26	p	compac

Block quotes

Use the `>` to make block quotes:

```
1 > This is what a block quote looks like.
```

This is what a block quote looks like.

Incremental content

Use `. . .` on its own line (xaringan's `---`) to reveal the rest of the slide on the next click:

```
1 Everyone sees this first.  
2  
3 . . .  
4  
5 Then this appears.  
6  
7 . . .  
8  
9 And finally this.
```

Everyone sees this first.

Then this appears.

And finally this.

Github code chunk highlighting

```
1 foo <- function(arg1 = 100, arg2 = "character string") {  
2   if (TRUE) {  
3     x = NULL # if, function, NULL are keywords a  
4     for (i in 1:10) x = c(x, mean(3 * rnorm(100) + 1))  
5   }  
6 }
```

Line highlighting

Highlight lines with the `code-line-numbers` cell option:

Code

```
1 ```{r}
2 #| code-line-numbers: "4,5"
3
4 library(ggplot2)
5
6 ggplot(mtcars) +
7   aes(mpg, disp) +
8   geom_point() +
9   geom_smooth()
10 ```
```

Output

```
1 library(ggplot2)
2
3 ggplot(mtcars) +
4   aes(mpg, disp) +
5   geom_point() +
6   geom_smooth()
```

Sizing text and code

One system for everything: wrap any content in a `.fontNN` div to scale it to NN% of the slide's base size (5% steps, `font10–font200`). It works the same on paragraphs, lists, and code chunks — a chunk and its printed output always match. For inline text there's also `[text]{.fontNN}` plus the `.small` and `.large` shortcuts.

Normal size:

```
1 head(mtcars[, 1:4], 3)
```

```
#>
#>      mpg cyl  disp  hp
#> Mazda RX4      21.0   6  160 110
#> Mazda RX4 Wag  21.0   6  160 110
#> Datsun 710      22.8   4  108  93
```

Wrapped in `::: {.font70}`:

```
1 head(mtcars[, 1:4], 3)
```

```
#>
#>      mpg cyl  disp  hp
#> Mazda RX4      21.0   6  160 110
#> Mazda RX4 Wag  21.0   6  160 110
#> Datsun 710      22.8   4  108  93
```

Layouts!

Tabset panels!

R Code Plot

```
1 ggplot(mtcars, aes(x = mpg, y = hp)) +  
2   geom_point() +  
3   theme_bw() +  
4   labs(color = "Cylinders")
```

Three (or more) equal columns

`.col`, no `width`

Any number of `.col` divs in a row auto-splits the space evenly — no separate class for 2 vs 3 vs 4 columns.

`.col`, no `width`

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

`.col`, no `width`

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Two equal columns

`.col`

Two consecutive `.col` divs with no `width` split 50/50 automatically. No outer `:::` `{.columns}` wrapper needed either — just write the columns.

`.col`

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Any split you want

```
.col width="65%"
```

`width` takes any percentage — 65/35, 42/58, whatever the content needs. No fixed set of splits to remember.

```
.col width="35%"
```

Only one side needs a `width`; a column left without one just takes whatever space remains.

Uneven columns, custom gap & alignment

gap and valign

Set `gap` or `valign` on any `.col` in the row and it applies to the whole row — handy here since this column's text is shorter than the code beside it, so `valign="middle"` centers them against each other.

```
1 foo <- function(x) {  
2   x + 1  
3 }
```

```
1 bar <- function(y) {  
2   y * 2  
3 }
```

Cards!

Grids of icon + text boxes

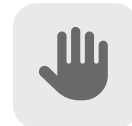
Card grids



Plan

Make a plan before doing big work.

read-only



Manual

Approve or deny each edit.

safe but slow



Accept Edits

Auto-applies file edits.

semi engagement



Auto

Approves everything.

use with care

Just write the boxes

Consecutive `::: {.card}` divs are grouped into a grid automatically — same idea as `.col`, no wrapper to write. The previous slide is just:

```
1  ::: {.card .icon-left color="dodgerblue"}
2
3
4  ### Plan
5
6  Make a plan before doing big work.
7
8  [read-only]{.tag}
9  :::
10
11 ::: {.card .icon-left color="gray"}
12
13 ...
14 :::
```

An icon on the card’s **first line** becomes the tile — from any icon library, since the shortcode has already expanded by the time lexis sees it.

Attribute	Effect
first line	, <code>{{< bi map >}}</code> , an emoji, <code></code>
icon=	Shorthand for an image file or an emoji
image=	A cropped photo band across the card top
color=	Accent color: a palette name or any CSS color
badge=	Small accent flag in the top-right corner
cols=	Cards per row (default: up to 4 in one row)
gap=	Gutter between cards
Class	Effect
.icon-left	Icon beside the heading instead of above it
.tint	Paint the accent color through an SVG icon <i>file</i>

Eight cards, `cols="4"`



Write code



Debug



Analyze data



Generate images



Summarize docs



Write & edit



Automate tasks

TODAY'S FOCUS



Build HTML

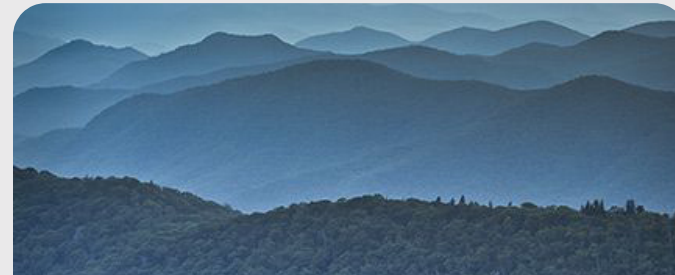
Cards with images: `image=`



Blue Ridge

`image=` crops a photo band across the top of the card, bled out to the rounded corners.

`morning`



Sunset

The band is `object-fit: cover`, so any aspect ratio fills it without squashing.

`evening`



Emoji work too

An `icon=` that isn't a file path is set as text in the tile.



So do SVG files

`.tint` paints the accent color through the file, so one gray icon set works with any color.

Cards on an inverse slide



Plan

Same accent, mixed toward black instead of white.



Auto

No extra classes needed — the card reads the slide.



Edit

`.highlight` still marks one out.

Full image background

```
1 {{< bg-image "images/blue_ridge_mountains.jpg" >}}
```

Full background color

```
1 {{< bg-color "#909099" >}}
```

Images!

Images have no border by default

This code produces the image on the right:

```
1 
```



Add a thin border with `.border`

This code produces the image on the right:

```
1 ::: {.border}
2 
3 :::
```



Or modify the border: `.borderthick`

This code produces the image on the right:

```
1 ::: {.borderthick}  
2   
3 :::
```



Or modify the border: `.whiteborder`

This code produces the image on the right:

```
1 ::: {.whiteborder}  
2   
3 :::
```



Or modify the border: `.whiteborderthick`

This code produces the image on the right:

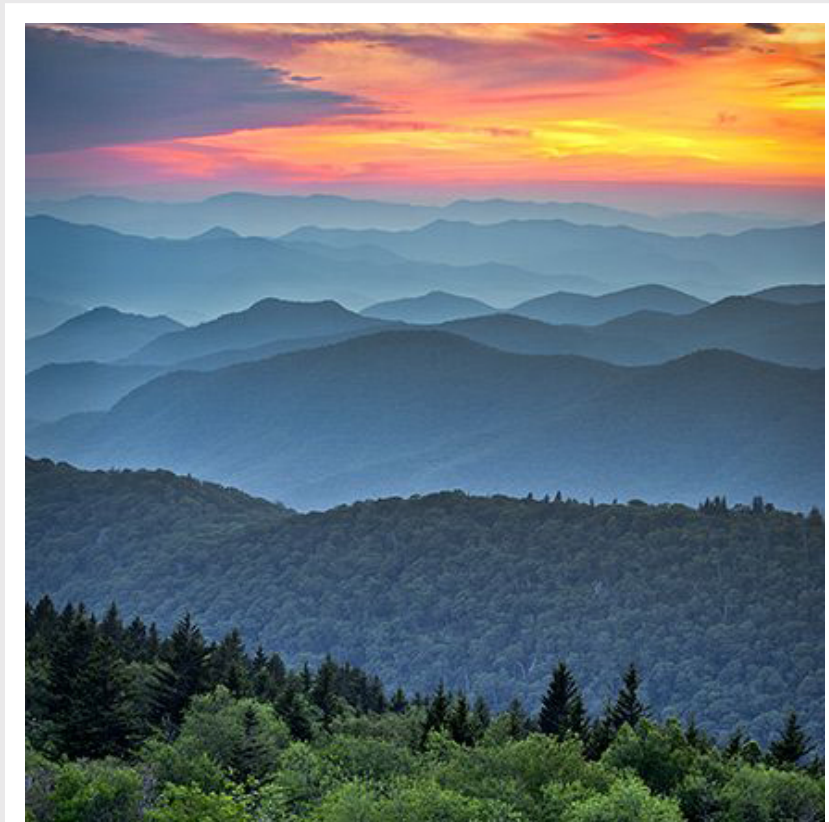
```
1 ::: {.whiteborderthick}  
2   
3 :::
```



Make a polaroid image: `.polaroid`

This code produces the image on the right:

```
1 ::: {.polaroid}
2 
3 :::
```



Make a circle image: `.circle`

This code produces the image on the right:

```
1 ::: {.circle}  
2   
3 :::
```



Make a thumbnail image: `.thumbnail`

This code produces the image on the right:

```
1 ::: {.thumbnail}  
2   
3 :::
```

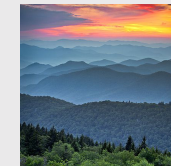
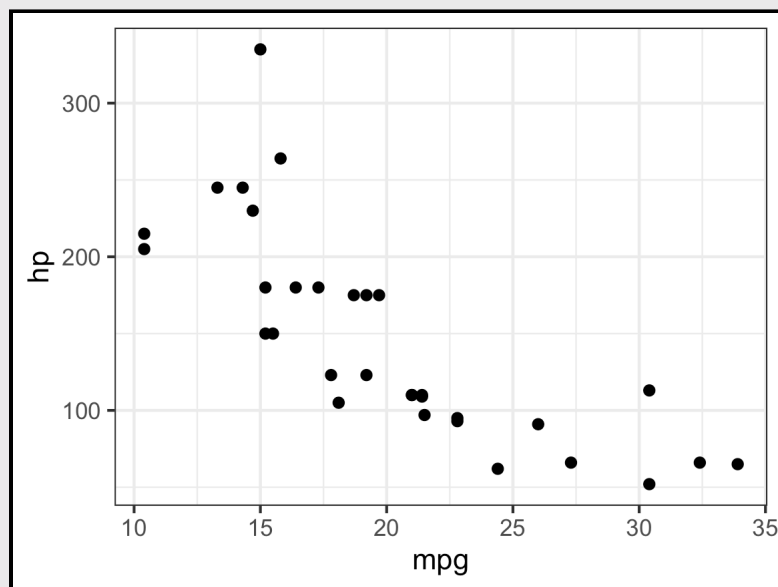
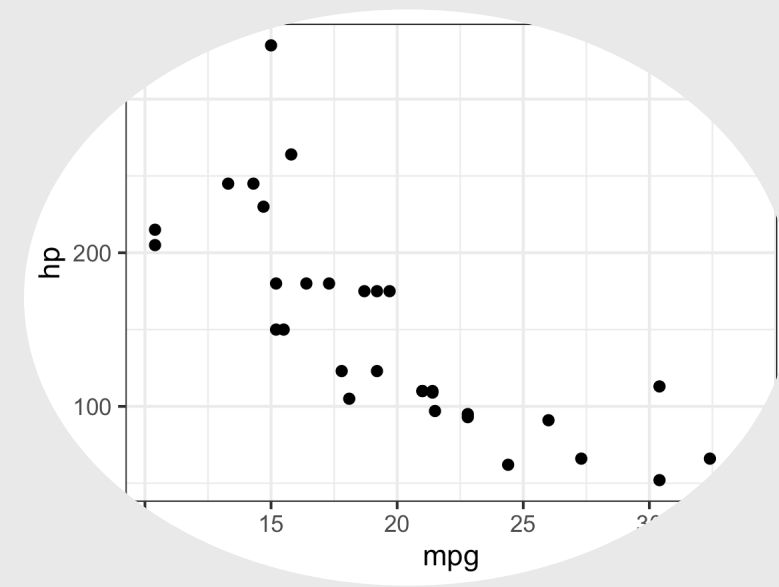


Image classes work on rendered charts too

```
1 ::: {.border}
2
3 ```{r}
4 ggplot(mtcars, aes(mpg, hp)) +
5   geom_point()
6 ```
7
8 :::
```



```
1 ::: {.circle}
2
3 ```{r}
4 ggplot(mtcars, aes(mpg, hp)) +
5   geom_point()
6 ```
7
8 :::
```



Thanks!

jhelvy.com 

jph@gwu.edu 

[@jhelvy_](#) 